

Step 04 — RBAC (manual test guide)

Goal: Confirm role → permission mapping, reusable RBAC middleware, tenant membership checks, and role-aware UI.

Maps to: Phase 1 — Foundation

Do not start Step 05 until every required item passes.

Artifacts:

- Markdown: docs/manual-test-guides/step-04-rbac.md
- PDF: docs/manual-test-guides/step-04-rbac.pdf (optional export)
- Postman: docs/postman/FMS-API.postman_collection.json (RBAC folder)

Prerequisites

- ☐ Steps 01–03 already passing
- ☐ PostgreSQL running; migrations applied (npm run prisma:migrate)
- ☐ Super Admin bootstrap env set in server/.env.local
- ☐ At least one company with a **company admin** created (Step 03)
- ☐ A self-registered **normal user** exists (Step 02 register)

Setup commands

Server:

```
cd finance-management/server
npm install
npx prisma generate
npm run prisma:migrate
npm run dev
```

Client (new terminal):

```
cd finance-management/client
npm install
npm run dev
```

Role → permission map (reference)

Role	Permissions
SUPER_ADMIN	tenants:manage
COMPANY_ADMIN	finance:write, reports:read, users:manage
NORMAL_USER	reports:read

Tenant-scoped probes require a company membership (tenantId not null).

Checklist

A. RBAC profile API

#	Test	Expected	Pass?
A1	GET /api/v1/rbac/me as Super Admin	HTTP 200; permissions includes tenants:manage	[]
A2	GET /api/v1/rbac/me as company admin	HTTP 200; includes finance:write, reports:read, users:manage	[]
A3	GET /api/v1/rbac/me as normal user	HTTP 200; includes only reports:read	[]
A4	GET /api/v1/rbac/me without cookie	HTTP 401	[]

Example:

```
curl -s -b cookies.txt http://localhost:4000/api/v1/rbac/me | jq
```

B. Permission probes (server enforcement)

Use cookies from the correct role for each row.

#	Test	Expected	Pass?
B1	GET /api/v1/rbac/probes/tenants-manage as Super Admin	HTTP 200	[]
B2	Same probe as company admin	HTTP 403 FORBIDDEN	[]
B3	GET /api/v1/rbac/probes/finance-write as company admin	HTTP 200	[]
B4	Same probe as normal user	HTTP 403 FORBIDDEN	[]
B5	GET /api/v1/rbac/probes/reports-read as company admin	HTTP 200	[]
B6	Same probe as normal user	HTTP 200	[]
B7	Finance or reports probe as Super Admin (no tenant)	HTTP 403 TENANT_REQUIRED	[]

C. Existing routes still guarded

#	Test	Expected	Pass?
C1	GET /api/v1/tenants as Super Admin	HTTP 200	[]
C2	GET /api/v1/tenants as company admin	HTTP 403	[]
C3	GET /api/v1/tenants unauthenticated	HTTP 401	[]

D. Client role-aware UI

#	Test	Expected	Pass?
D1	Home as Super Admin	Shows Manage companies + Role & access	[]

D2	Home as company admin	Shows Finance write enabled ; no Manage companies	[]
D3	Home as normal user	Shows Read-only reports access	[]
D4	Open <code>/access</code> signed in	Permissions list matches API <code>/rbac/me</code>	[]
D5	Run Access probes on <code>/access</code>	Each probe status matches role expectations	[]
D6	Normal user opens <code>/tenants</code>	Redirected home	[]

E. Coding rules smoke

#	Test	Expected	Pass?
E1	Permission map lives in config	<code>server/src/config/permissions.ts</code>	[]
E2	Middleware reused	<code>requireRoles</code> , <code>requirePermission</code> , <code>requireTenant</code>	[]
E3	Super Admin guard uses shared middleware	<code>requireSuperAdmin</code> → <code>requireRoles("SUPER_ADMIN")</code>	[]
E4	Tenant isolation helper	<code>server/src/utils/tenantAccess.ts</code> (<code>assertSameTenant</code>)	[]

Pass criteria for Step 04

- ☐ A1–A4 pass
- ☐ B1–B7 pass
- ☐ C1–C3 pass
- ☐ D1–D6 pass
- ☐ E1–E4 pass

When all required items pass: mark Notion **Implement RBAC...** Done, then start **Step 05 — Audit foundation**.

Notes / failures

Item	What failed	Fix applied

Optional PDF

Printable copy of this guide:

<docs/manual-test-guides/step-04-rbac.pdf>